

MODULE 11 · DOMAIN 3 - CLAUDE CODE CONFIGURATION & WORKFLOWS

Claude Code in CI/CD

Headless mode, the flags that matter, and why nobody is there to notice when it goes wrong.

DOMAIN 3 of 5 EXAM WEIGHT 20% MODULE 11 of 20

The three jobs a human was doing

- ◆ **Granting permission.** Approving each action, or refusing it.
- ◆ **Judging completion.** Deciding whether the work is actually finished and any good.
- ◆ **Watching for trouble.** Noticing a loop, a wrong turn, a runaway cost.
- ◆ **Remove the human and all three become configuration.** If you have not configured one, it is not happening.

ON THE EXAM

Every flag in this module replaces one of these three. That is a useful way to remember which does what.

The flags that matter

FLAG	WHAT IT DOES
<code>-p</code>	Print mode. Runs once, non-interactively, and exits.
<code>--output-format json</code>	A machine-readable object: result, cost, duration, session ID, stop reason.
<code>--max-turns N</code>	A hard ceiling on agentic steps.
<code>--max-budget-usd</code>	A hard ceiling on spend for this invocation.
<code>--allowedTools</code>	The tool surface. What it cannot call, it cannot do.
<code>--append-system-prompt</code>	Adds instructions to the default prompt.

ON THE EXAM

Piped stdin becomes context before the prompt. That is how a diff gets in.

Why the JSON output matters

- ◆ **Text output is prose.** Good for a human, bad for a parser, and worse for a decision.
- ◆ **JSON gives you the result plus the metadata,** including cost and the stop reason.
- ◆ **The stop reason is how your pipeline classifies the ending.** Module 4's lesson, in a pipeline.
- ◆ **Capture the session ID here** if a later stage needs to resume.

ON THE EXAM

A pipeline that reads only the prose has thrown away everything it needed to know about how the run ended.

Append, or replace?

-- APPEND - SYSTEM - PROMPT

Adds to the default

- Claude Code's built-in instructions stay
- Tool usage guidance, safety, environment context
- Your text is added on top
- Almost always what you want

-- SYSTEM - PROMPT

Replaces the default entirely

- The built-in instructions are gone
- Including the ones you did not know were there
- You are now responsible for all of it
- For deliberate, tested customisation only

THE TIEBREAKER

The two replace flags are mutually exclusive. Append can combine with either. Reach for append by default.

Where CLAUDE.md sits, precisely

- ◆ CLAUDE.md is added as a user message, following the default system prompt.
- ◆ `--append-system-prompt` appends to the system prompt itself.
- ◆ Neither one edits Claude Code's default prompt. Only `--system-prompt` does that.
- ◆ CLAUDE.md is read on every run, so it is a real cost in a pipeline that runs on every pull request.

ON THE EXAM

Three mechanisms, three different positions in the request. Questions exploit people assuming they are interchangeable.

Caps, and the one with no default

- ◆ In print mode, `--max-turns` has no default limit. Omit it and there is no ceiling.
- ◆ Three to eight turns suits most review tasks. Enough to read and reason, not enough to wander.
- ◆ `--max-budget-usd` caps spend per invocation, which is the backstop when turns are cheap but long.
- ◆ Add a workflow timeout as a second layer. Two independent limits beat one.

ON THE EXAM

Cost overruns usually come from one badly scoped task needing far more iterations than expected, not from run volume.

Permissions with nobody to ask

- ◆ **PermissionRequest hooks do not fire in plain headless runs.** There is nobody to prompt.
- ◆ **So use `--allowedTools`, and `PreToolUse` hooks,** which do fire.
- ◆ **A read-only review needs `Read`, `Grep` and `Glob`.** Nothing that writes, nothing that runs shell.
- ◆ **`--dangerously-skip-permissions` belongs only in an isolated, ephemeral container,** and never on a machine that matters.

ON THE EXAM

The permission prompt was a control. Removing the human removes the control unless you replace it deliberately.

A run can exit zero, produce nothing useful,
and be recorded as a **success**.

Classifying the ending in a pipeline

- ◆ **Read the stop reason from the JSON**, not the exit code alone.
- ◆ **Hitting `--max-turns` is an abort, not a completion.** The review is partial and should not be posted as final.
- ◆ **An empty result with a zero exit is a failure**, and your pipeline must treat it as one.
- ◆ **Fail the stage, or flag for human review.** Never publish a partial result as a finished one.

ON THE EXAM

The same three endings from module 4: finished, capped, errored. A pipeline that knows only pass and fail collapses them.

Who should review the code?

SAME SESSION

The author reviews itself

- Holds every assumption it made while writing
- Sees the code it meant to write
- Agrees with its own reasoning
- Misses exactly what it misunderstood

FRESH SESSION

A reader with no history

- Sees only the diff and the standards
- No memory of why a choice was made
- Reads the code as written, not as intended
- Catches assumptions the author cannot see

THE TIEBREAKER

A fresh session is the closest thing to a second pair of eyes. Its ignorance of the reasoning is the whole point.

Reducing review noise

- ◆ A reviewer that flags everything gets ignored, and then the real findings are ignored too.
- ◆ Give it the standards, not just the diff. Your conventions, in a file it reads.
- ◆ State explicitly what is out of scope, so style opinions do not arrive as security findings.
- ◆ Ask for severity, and post only above a threshold. Let the pipeline filter, not the model's judgement about relevance.
- ◆ Several focused reviews beat one general one on a large codebase.

ON THE EXAM

Every noisy comment on a PR trains the team to skim. That cost lands on the findings that mattered.

Secrets and safety in a pipeline

- ◆ The API key comes from a repository secret, never hardcoded and never on the command line.
- ◆ A key on the command line appears in the process list and the logs.
- ◆ Deny reading environment files, so credentials in the runner never reach the model.
- ◆ Scrub sensitive environment variables from the subprocess context.
- ◆ A CI runner is full of credentials. Assume the agent can see anything you have not excluded.

ON THE EXAM

Everything from module 8's permissions applies here, and the consequences are larger because the environment is richer.

Currency check: --bare

TOPIC	EXAM ANSWER	CURRENT REALITY
Config in CI	CLAUDE.md and hooks load as normal	--bare skips auto-discovery of hooks, skills, plugins, MCP servers and CLAUDE.md.
Why	Not covered	Determinism. A teammate's local hooks cannot change a pipeline run.
Auth under --bare	Not covered	Must come from an API key or key helper, not the keychain.

ON THE EXAM

Anthropic recommends --bare for scripted calls. It is not examinable, but it is the right answer at work.

Reading the stem: CI questions

WHAT THE STEM SAYS	WHAT IT WANTS
"the pipeline cannot parse the result"	--output-format json
"costs spiked on one pull request"	--max-turns, and --max-budget-usd
"it started modifying files during a review"	--allowedTools, restricted to read tools
"reviews are posted even when incomplete"	Classify the stop reason before publishing
"the review misses the bugs it introduced"	A fresh session, not the authoring one
"developers ignore the review comments"	Severity threshold, scope limits, team standards

ON THE EXAM

Map each one back to the three human jobs: permission, completion, or watching for trouble.

Module 11 in five lines

- ◆ Headless removes the human. Permission, completion and vigilance all become configuration.
- ◆ `--output-format json` gives you the stop reason. The exit code alone does not.
- ◆ `--append-system-prompt` adds; `--system-prompt` replaces everything, including what you relied on.
- ◆ In print mode `--max-turns` has no default. Hitting it is an abort, not a completion.
- ◆ A fresh session reviews better than the session that wrote the code.

ON THE EXAM

That closes Domain 3. Five scenarios, then Domain 4 and prompt engineering.

A team runs an automated review on every pull request with `claude -p`, posting the result as a PR comment.

On large pull requests the comment sometimes stops mid-sentence, or covers only the first two files. The workflow reports success every time. The JSON output shows `stop_reason` indicating the turn cap was reached.

The turn cap is set to 3.

What should they change?

- ☐ A Remove `--max-turns` so large reviews can run to completion
- ☐ B Raise the cap, and have the workflow check the stop reason before posting
- ☐ C Switch to `--output-format text` so the comment reads more naturally
- ☐ D Split large pull requests into smaller ones

◆ PAUSE AND DECIDE

What should they change?

- A

 Remove the cap
In print mode there is no default, so removing it means no ceiling at all. It trades truncation for unbounded cost.
- B

 Raise it, and check the stop reason
Fixes the truncation and the silent success separately, which is right because they are two faults.
- C

 Text output
Discards the stop reason, which is the only reason they can currently diagnose this at all.
- D

 Smaller pull requests
Sensible practice, and it does not stop a capped run being reported as a complete review.

THE SIGNAL

Two faults in one stem. A cap set too low, and a workflow that publishes a capped run as finished. Only one option addresses both.

An engineer wants the CI reviewer to behave as a security specialist. They add `--system-prompt` "You are a security engineer. Review the diff for vulnerabilities and report findings as JSON."

After the change, review quality drops sharply. Claude ignores the repository's conventions, occasionally attempts to edit files during a review, and its file navigation becomes erratic. The prompt text itself is correct and well written.

What went wrong?

- ☐ A The prompt needs more detail about the repository's conventions
- ☐ B `--system-prompt` replaced Claude Code's default instructions, including tool guidance and safety context
- ☐ C The model is too small for security review work
- ☐ D CLAUDE.md is not being loaded in headless mode

◆ PAUSE AND DECIDE

What went wrong?

A More detail needed

True in a sense, and it treats the symptom. They would be rewriting a default prompt they deleted by accident.

C Model too small

Nothing changed about the model. The behaviour changed when the flag did.

B The default was replaced

Erratic navigation, ignored conventions and attempted edits are all the built-in guidance going missing at once.

D CLAUDE.md not loading

Would explain the conventions, not the erratic navigation or the attempted edits.

THE SIGNAL

Three unrelated symptoms appearing together is the signature of removing a default, not of a prompt that needs improving.

A pipeline generates code for a feature, then in the same session asks Claude to review its own work against the team's standards before opening a pull request.

The reviews are consistently positive. Human reviewers subsequently find issues the automated review did not mention, and several of them stem from a misreading of an existing interface.

The team wants the automated review to catch these.

What is the most effective change?

- ☐ A Give the review step a longer, more detailed checklist
- ☐ B Run the review in a separate session that sees only the diff and the standards
- ☐ C Increase max-turns for the review step so it can look more carefully
- ☐ D Ask the review step to be more critical and assume the code is wrong

◆ PAUSE AND DECIDE

What is the most effective change?

A Longer checklist

A checklist cannot catch a misreading. The reviewer shares the misreading and will tick the box confidently.

C More turns

More time spent agreeing with itself. Duration was never the constraint.

B Separate session

The misunderstanding lives in the authoring context. A reviewer without that context reads the code as written.

D Be more critical

Adjusts tone, not information. It will find different things to say about code it still misunderstands.

THE SIGNAL

"Stem from a misreading of an existing interface" is the planted detail. The reviewer holds the misreading, so it cannot see it.

An automated reviewer posts between fifteen and forty comments per pull request. Most are stylistic preferences, naming suggestions and observations about code the pull request did not touch.

Developers have started collapsing the bot's comments without reading them. Last month a genuine SQL injection finding was missed this way.

The team wants the reviewer to be read again.

What is the most effective change?

- ☐ A Have the reviewer assign a severity to each finding, and post only high and critical ones
- ☐ B Reduce max-turns so it produces fewer comments
- ☐ C Ask the reviewer to be less pedantic in the prompt
- ☐ D Post the full review as a file attachment instead of comments

◆ PAUSE AND DECIDE

What is the most effective change?

- A

 Severity, filtered in the pipeline
The filtering happens in code against a threshold, so a critical finding cannot be crowded out by naming suggestions.
- B

 Fewer turns
Truncates the review arbitrarily. You lose findings by position, not by importance.
- C

 Be less pedantic
Moves the filtering into the model's judgement about what matters, which is the thing that is currently wrong.
- D

 An attachment
Same volume, now behind an extra click. It makes the ignoring easier.

THE SIGNAL

Also worth adding: scope it to changed files, and give it the team's standards. Option A is the one that fixes the signal-to-noise ratio structurally.

A nightly job runs `claude -p` in a GitHub Actions container to triage failing tests. It is invoked with `--allowedTools Read,Grep,Glob,Bash`.

A security review raises two concerns. The runner's environment contains deployment credentials, and the job has previously run a Bash command that read a `.env` file into its context.

The job genuinely needs to run the test suite.

What should they do?

- ☐ A Remove Bash from `--allowedTools`
- ☐ B Deny reading `.env` files, scrub sensitive environment variables from the subprocess, and keep Bash scoped to the test command
- ☐ C Add a `CLAUDE.md` rule instructing Claude never to read credential files
- ☐ D Use `--dangerously-skip-permissions` since the container is ephemeral

◆ PAUSE AND DECIDE

What should they do?

- A

 Remove Bash
Solves the leak by removing the capability the job exists to use. The stem says it needs to run tests.
- B

 Deny, scrub, and scope
Three layers, each enforced, and the job keeps the capability it needs.
- C

 A CLAUDE.md rule
Context. It cannot prevent a Bash command from reading a file.
- D

 Skip permissions
Backwards. Ephemeral limits the blast radius; it does nothing about credentials already in the environment.

THE SIGNAL

Ephemeral is not the same as safe. The credentials are in the runner while the job runs, which is when the leak happens.